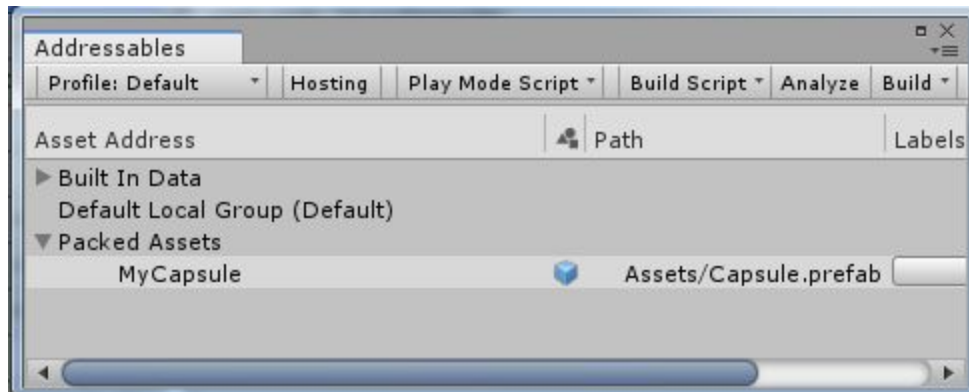


Unity Addressable Assets System 簡介



環境要求:

AAS version 0.4.8

Unity version 2018.3.0f2

[#1. 概要](#)

[#2. 如何安裝](#)

[#3. 如何使用](#)

[@1. 建立AssetGroup](#)

[@2. 給資源一個別名, Asset Address](#)

[@3. 程式如何加載?](#)

[@4. 關鍵API補充說明](#)

[各API的AssetAddress值為什麼是object型態?](#)

[LoadAsset](#)

[Instantiate](#)

[LoadAssets](#)

[InstantiateAll](#)

[@5. 相同別名處理規則, 補充說明](#)

[#4. 進階補充](#)

[@0. AddressableAssetSettings.asset](#)

[@1. Profile](#)

[@2. 用Label批量加載](#)

[@3. Build Path & Load Path](#)

[@4. 如何出APK\(第一次\)](#)

[@5. 如何刷新先前打包過的資料](#)

[@6. AssetReference跟AssetLabelReference](#)

[請視專案情況決定是否使用這樣的方式](#)

[@7. 三個Play Mode介紹](#)

[@8. Hosting Services](#)

[@9. Analyze](#)

[@10. Clean](#)

[@11. Built In Data](#)

[@12. Addressables Profiler視窗](#)

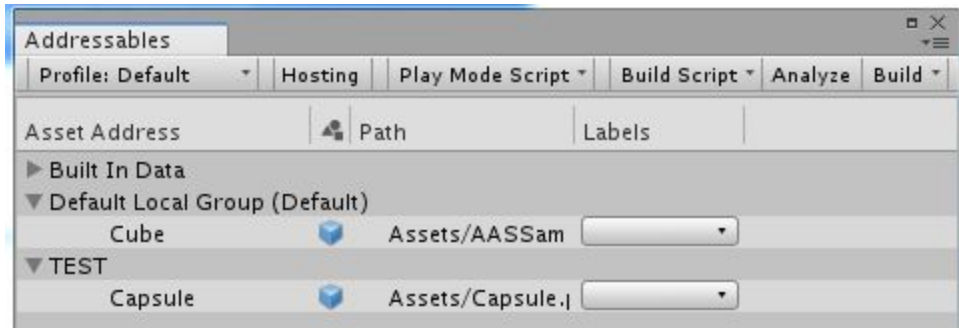
[#5. 延伸閱讀](#)

[@如何寫一個可以動態刷新資源的Profile\(簡單範例, 不說明\)](#)

[@如何讓Hosting Service能被其他機器讀取?](#)

[@其他問題](#)

#1. 概要



本文主要介紹 [Unity Addressable Assets System](#), 以下簡稱AAS
是Unity在2018年推出的新的資源管理系統
預設仍是使用Unity 的Asset Bundle機制為基底, 但允許自定義, 後面會說到

AAS主要想處理掉幾個, 每個專案開發都會反覆遇到的問題

1. 程式人員跟其他職能, 最常碰撞在一起的, 檔案路徑&名稱問題
2. AssetBundle或衍生的Library, 學習門檻陡峭問題
3. 讓開發者告別始終戒不掉的Resources
4. Runtime記憶體優化, 獨立把資源的記憶體拉出來, 簡單明瞭看出資源問題

如果要用一句話解釋什麼是AAS?

[AAS就是一個資源的\[打包/加載/管理\]系統](#)

而AAS最核心的概念就是:

[Name a thing, Load a thing](#)

給資源一個別名, 用這個別名去加載他

閱讀本文前, 推薦先閱讀, 本文將假設, 讀者已了解這些內容

1. [Package Manager簡介](#)
2. [Asmdef簡介](#)

#2. 如何安裝

請注意, 以下內容基於Unity 2018.3撰寫, 用舊的版本請自己Google一下舊版安裝方式

首先, 打開一個Unity專案, 接著打開上方選單的
[Window/Package Manager](#)

請找到, Packages視窗的[Advanced/Show preview packages](#), 將選項打勾
找到[Addressables System](#), 看到右邊的下拉選單資訊, 如果是0.5.3, 請改為0.4.8

理由是, 我認為0.5.3問題太多, 不推薦使用, 但不可否認, 0.5.3正在朝相對友善的方向前進, 只能期待下一版了
總之, 下面的教學都是針對0.4.8的, 如果想使用0.5.3需要自行摸索, 或者直接來問我用法, 0.5.3在PC上算堪用了



選好0.4.8後按下Install等待安裝跑條

接著請下載這個[Bug修正程式碼](#), 丟到任意Editor資料夾下

OR

直接下載並安裝這個[Sample unitypackage](#), 該package內包含上述修正

然後到上方選單的

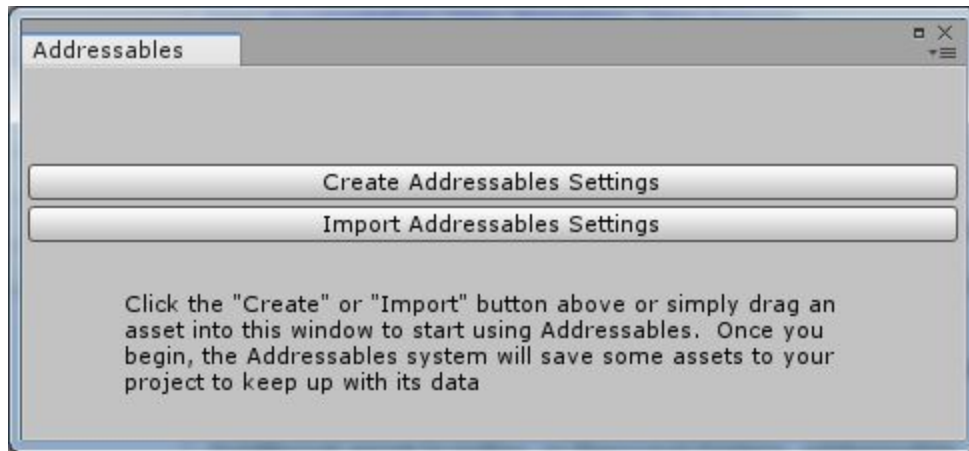
[Windows/Asset Management/046BugFix/AutoDirty/Enable](#)

將自動修正功能打開, 沒開這功能, AAS會在Unity重啟後故障

安裝完成後, 接著要進行AAS啟用的動作

請找到上方選單的

[Windows/Asset Management/Addressable Assets](#)



打開[Addressables](#)視窗, 這是AAS的主要視窗

第一次打開視窗, 應該會看到兩個按鈕

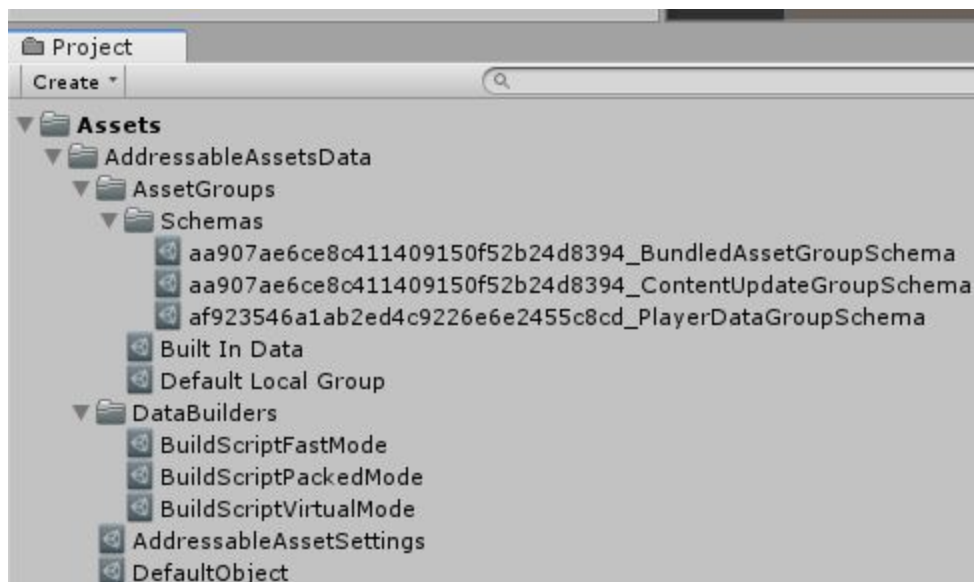
請按下[[Create Addressables Settings](#)]進行AAS的啟用動作

這會建立一個名為[AddressableAssetsData](#)的資料夾

所有AAS的設置, 都會存在這個資料夾內

由於本文是基礎教學, 這裡就不對這個檔案夾內容, 做出太過詳細的說明了

對初學者來說就是, 上傳版控資料時, 這個資料夾下的都要記得傳, 這樣就夠了



#3. 如何使用

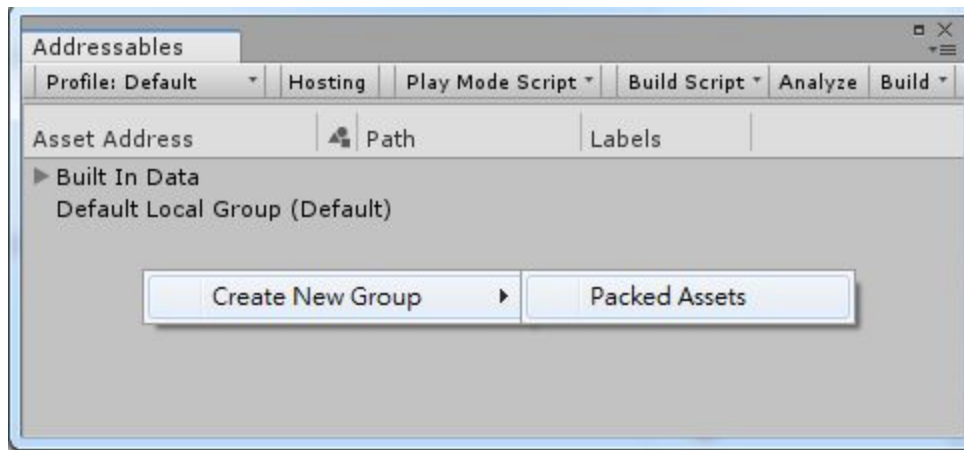
再次強調, 本文的主題是AAS的最基礎使用教學, 所以這裡只教了最簡易的使用方式
進階的使用方式教學, 請參考延伸閱讀各文件說明

@1. 建立AssetGroup

首先打開Addressables視窗, 在視窗空白處使用右鍵選單的

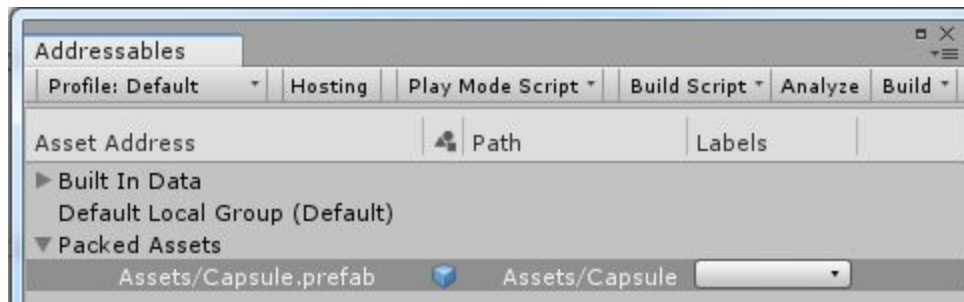
[Create New Group / Packed Assets](#)

新增一個AssetGroup, 要不要改名可以自行決定, 操作方式與一般檔案總管一致, 不再贅述



AssetGroup可以想成, 是一個AssetBundle包(或者想成是一個.zip壓縮檔)

你可以在[Project視窗](#), 直接把你想登記到AAS的資源, 直接拖曳放入你想放入的AssetGroup中



同一個Asset, 同一時間只會存在於一個AssetGroup

也就是說, 當你原本已經登記在Group1的A資源, 再次登記到Group2時
AAS會自動幫你把A資源移籍到Group2

@2. 給資源一個別名, Asset Address

每個剛登記到AAS的資源, 預設都會使用從Assets開始算的檔案路徑, 作為預設別名
把資源登記進AAS後, 你可以選擇用這個路徑作為別名去加載這個資源
也可以選擇重新給他一個簡短一點的別名, 總之, 我們的程式就只會認這個別名
去加載這個資源



請注意, 如果不改預設別名, 當檔案路徑或名稱發生變化時
AAS是**"不會"**自動幫你修改這個預設別名的喔

@3. 程式如何加載?

打開程式碼, 在reference得到[UnityEngine.Addressables](#), 這個namespace的地方
調用[Addressables](#)類別的API就可以了

寫法大概像是這樣

```
using UnityEngine.AddressableAssets;  
  
Addressables.LoadAsset<GameObject>(PrefabName).Completed += operation =>  
{  
    Debug.Log($"LoadAsset > {operation.Status}");  
};
```

特別說明下, AAS的API, 預設全都是**非同步**加載的
官方給的理由是, 這樣可以讓程式人員在寫程式碼的時候
直接考量好所有加載情況(本地/遠端)
讓你的程式碼可以在不同的加載情況切換下, 依然能順利無縫運行

@4. 關鍵API補充說明

非程式人員可以跳過這段不看

各API的AssetAddress值為什麼是object型態?

因為除了string, AAS還收ResourceManager的ILocation
這部分, 涉及的範圍不是AAS了, 而是ResourceManager
所以不多做解釋

LoadAsset

這個API跟Resources.LoadAsync相同, 也就是說
他只會加載真的檔案給你, 不會幫你進行Instantiate的動作

當你把這個Asset加載進來之後, 請自己寫一個變數存他
否則之後你想使用ReleaseAsset API的時候, 會給不了參數

另外, 從Addressable Profiler視窗(目前是半成品)
有觀察到一個特性, 如果沒在用的Asset, 一段時間後會自己drop掉
總之, 記得存一下(我們自製的AssetManager套件, 有處理這個問題)

最重要的是, 透過這個API加載的Asset

請勿使用UnityEngine.Object.Instantiate自己實例化出來

理由是, 你這樣實例化出來的東西, AAS並不知道, 會造成各種控管的leak問題

Instantiate

這個API會幫你執行LoadAsset + Instantiate的動作
若資源執行LoadAsset後有Instantiate的需求, 請直接改用這個API
這樣產生出來的instance, AAS可以正確幫你進行記憶體管理

如果資源的產生到銷毀, 全都是使用AAS的API
AAS會同步自動釋放掉, 這個API自動載入的Asset

LoadAssets

這個API其中一種參數, 收的是IList<object>, 但請不要誤會
以為他能讓你一次輸入多個Asset Address字串
他這個List的多個值, 會連鎖使用到後面的MergeMode進行處理
詳細可以閱讀他原始碼MergeMode的說明, 或者找我拿中文解釋

InstantiateAll

key值, 所有擁有相同Asset Address值的資源, 實例都會一起被創建出來

@5. 相同別名處理規則, 補充說明

非程式人員可以跳過這段不看

AAS不會阻止相同別名, 也就是說, 如果AAS內有兩個不同的資源都被命名為, MyCube, 然後我們嘗試用MyCube去進行加載

這時, 我們會需要對這部分的處理規則, 進行延伸理解
這時如果用的是單體的API, 例如LoadAsset或Instantiate
則AAS會直接從他的資源裡, 把他翻到的第一個資源丟給你
你將無法精確決定加載出來的資源結果

如果這時你使用的是複數型API, LoadAssets跟InstantiateAll
則預設會全撈給你

關於別名, 另外還會有多重別名的問題, 有興趣可以自行閱讀AAS的原始碼
找到MergeMode這個Enum的註解, 有詳細說明, 個人不推薦使用
搞得太複雜了, 非程式應該會頭暈

如果有組合式加載的需求, 可以考慮用Label替代
詳細用法不在這邊贅述

#4. 進階補充

看完上一個章節, 就已經能最簡單的使用AAS了
這個章節會多多說一些設置基本知識點
可看可不看, 太複雜的依然不會提, 難搞的全都會整在#5

@0. AddressableAssetSettings.asset

AAS的主體設置全都在這個檔案上, 很多進階設置都在這個檔案上
你可以透過Addressables視窗左上的下拉選單, [Profile: xxxxx / Inspect Profile Settings](#)
快速選取到這個檔案

正常來說, 不是很熟悉AAS的使用者, 應該只會用到[Labels](#)區塊的功能
這個區塊是用來新增Label, 進行批量讀取的

這個檔案有個很坑人的地方, 就是每次選取這個檔案時
Profiles區塊的選單, 會回到第一組(Default), 導致改錯Profile
負責修改Profile的人員要多注意這個坑

@1. Profile

AAS的[Profile](#)會被所有的[BundledAssetGroupSchema](#)參考到
一個專案內可以有多組Profile, 但同一時間只會有一組Profile被啟用

要知道目前哪一組Profile被啟用, 很簡單, 看一下Addressables視窗左上角
[Profile: xxxx](#)就有顯示了, 切換也在這裡切換

Profile主要功能是, 管理打包跟加載的設定值
寫法需要比較多篇幅的文字才能說得清, 就不再贅述了

管理Profile的方式可以在[AddressableAssetSettings.asset](#)介面的[Profiles](#)區塊上找到

@2. 用Label批量加載

如果有批量加載的需求, 除了把Asset Address全設相同, 這個方式AAS還有第二個選擇, 每一個登記到AAS的Asset, 都有Labels這個欄位
這個欄位可以設置, 0 ~ N組Label, 設好Label之後
程式可以直接使用複數加載的API, 依Label一口氣全部加載

新增/刪除 Label的介面在[AddressableAssetSettings.asset](#)介面的Labels區塊可以找到

@3. Build Path & Load Path

每個掛載[BundledAssetGroupSchema](#)的AssetGroup
都可以看到兩個區塊, 一個是Build Path, 一個是Load Path

Build Path, 代表的是, 當AAS進行資源打包時, 打包出來的包體檔案, 要放在哪個地方

Load Path, 代表的是, 當AAS進行資源加載時, AAS要用什麼樣的路徑進行加載

一般情況下, 每個新建的AssetGroup, 應該都是預設為
[LocalBuildPath](#)以及[LocalLoadPath](#), 然後下面會出現一串路徑文字
這串路徑文字是, 參考當前所啟用的Profile, 用該Profile的設定值
進行計算所得出的路徑值

LocalBuildPath, LocalLoadPath, 等選單的來源是[AddressableAssetSettings.asset](#)檔
找到上面的Profiles區塊, 上面每一個[Profile Entries](#)都會成為下拉選單的選項

要提供可用的Profile設置, 會需要一些Profile設置的相關學習
內容蠻瑣碎的, 不再贅述, 想進一步學習請翻#5

@4. 如何出APK(第一次)

首先, 這裡我們使用的是0.4.8, 這點很重要, 因為每一版的打包方式都在改變

其次, 請打開Addressables視窗, 請確認至少有一個AssetGroup掛載了ContentUpdateGroupSchema, 並且將Static Content打勾
然後, 再確認, 這個AssetGroup內, 至少有一個Asset

再來, 要知道在包APK的時候, AAS會自動包資料的特性(053這個特性已經被拿掉了)
因此, 我們只需要依照正常的APK打包方式, 進行APK打包即可

請注意, 在打包之後, 會在apk所在的路徑上, 同步生成一個檔案
[addressables_content_state.bin](#)

這個檔案記錄了這次打包的一些必要資訊, 每次 build APK都會刷新他
所以, 使用AAS後, build 版本這件事, 會需要謹慎一點

特別說明:

到這裡為止, 我們使用的, 仍是Default的Profile
官方這個Profile預設填寫的方式, 並沒有打算
讓我們用到AAS另一項, 動態刷新資源包的能力
因此, 如果有需要使用到動態刷新資源包能力, 請到#5找教學吧

@5. 如何刷新先前打包過的資料

前題:

這段文件的說明, 建立於, 您已經不是使用Default Profile的情形下

所有後續新增的資源, 都可以透過Addressables視窗的上方選單功能
[Build/Build for Content Update](#)

進行資源重新打包

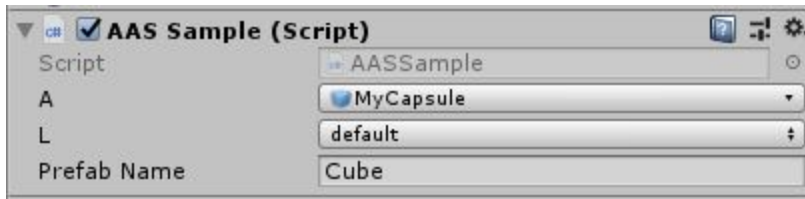
這個打包動作, 會自動視資源異動的情況, 進行相應的增量打包

若您已學會, 如何寫一個可用於動態刷新資源包的Profile
則只需要透過這個功能, 便可立即將新的資源刷入遊戲內

這個打包方式, 目前會有個小困擾, 就是, 他並不會去清除, 老舊的bundle包檔案
所以隨著打包次數增加, 存放bundle包的資料夾, 也會愈來愈胖, 請定期清空該資料夾
以免硬碟空間不足

@6. AssetReference跟AssetLabelReference

```
public AssetReference a;  
  
public AssetLabelReference l;
```



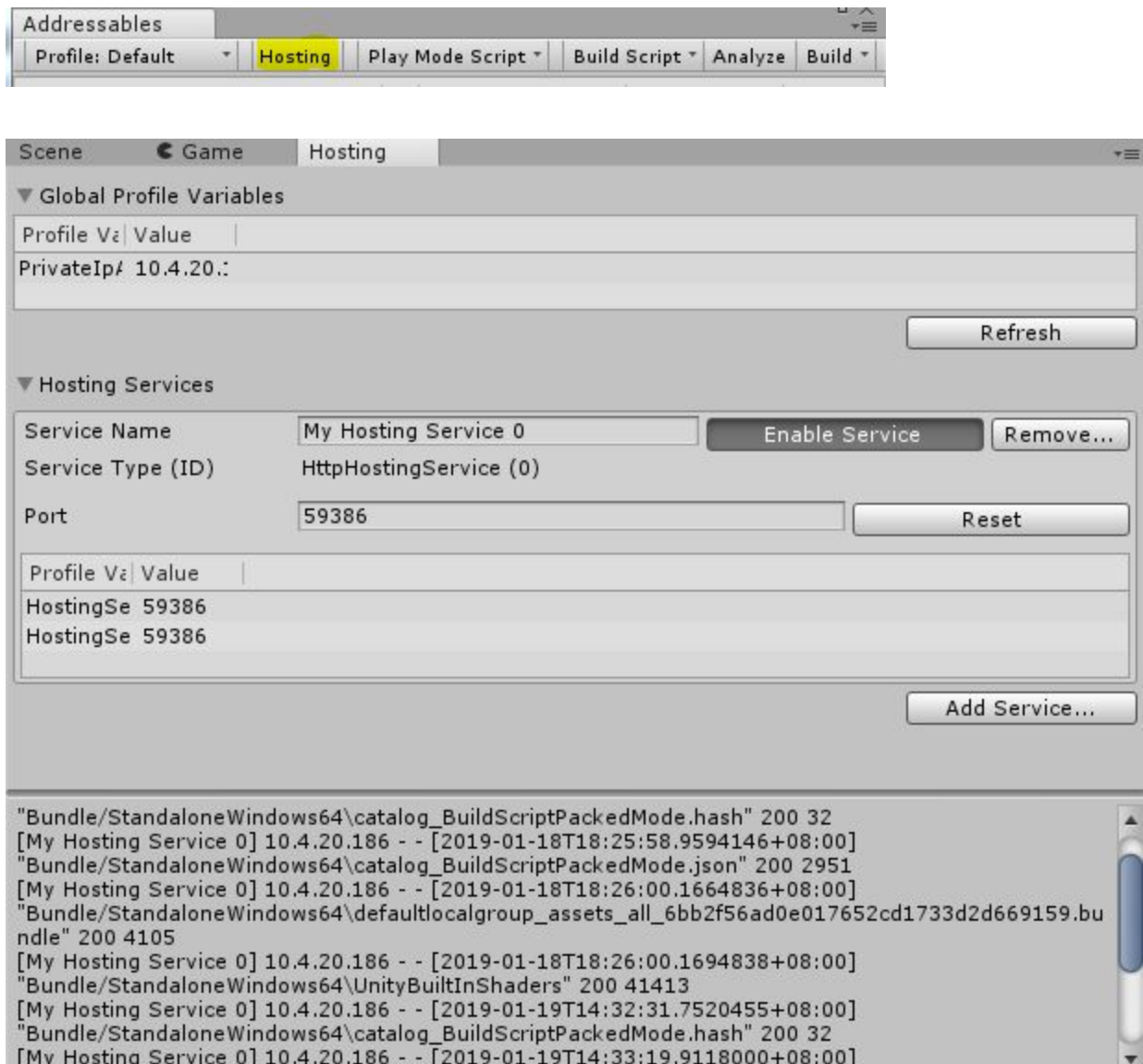
AAS本身有提供這樣子的設計, 使用上也相當簡便
但這樣的使用方式, 可能會讓開發的製程與Unity本身有更緊密的結合
請視專案情況決定是否使用這樣的方式

@7. 三個Play Mode介紹



位置在: Addressables視窗上方選單, [Play Mode Script](#)
其他詳細說明請參考[連結](#)

@8. Hosting Services



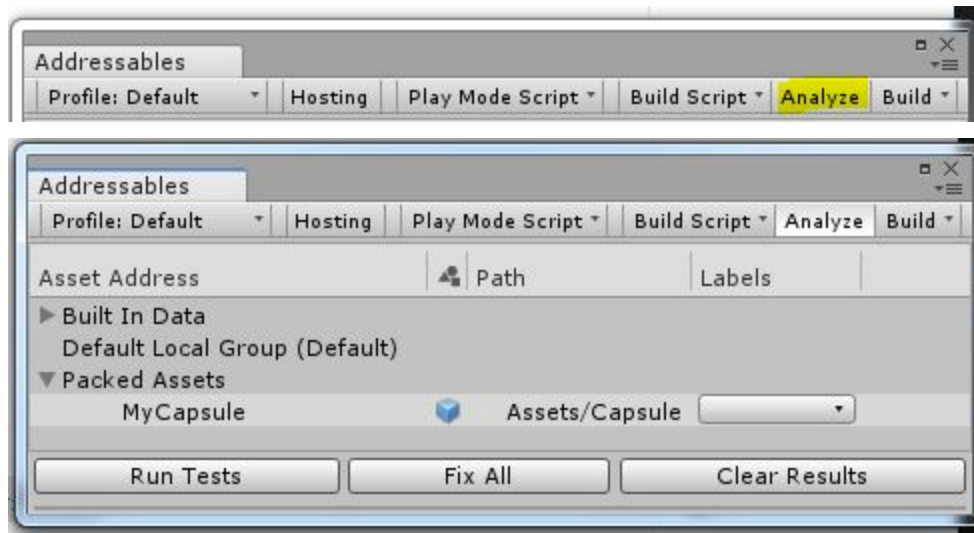
位置在: Addressables視窗上方選單, [Hosting](#)

簡而言之, 就是AAS內建的檔案伺服器功能

其他詳細說明請參考[連結](#)

或者[這篇](#)官方文件(英文)

@9. Analyze



位置在: Addressables視窗上方選單, [Analyze](#)
AAS內建的資料分析功能介面, 目前只提供了
AssetBundle常見的: [資源交叉引用分析&修正功能](#)

@10. Clean

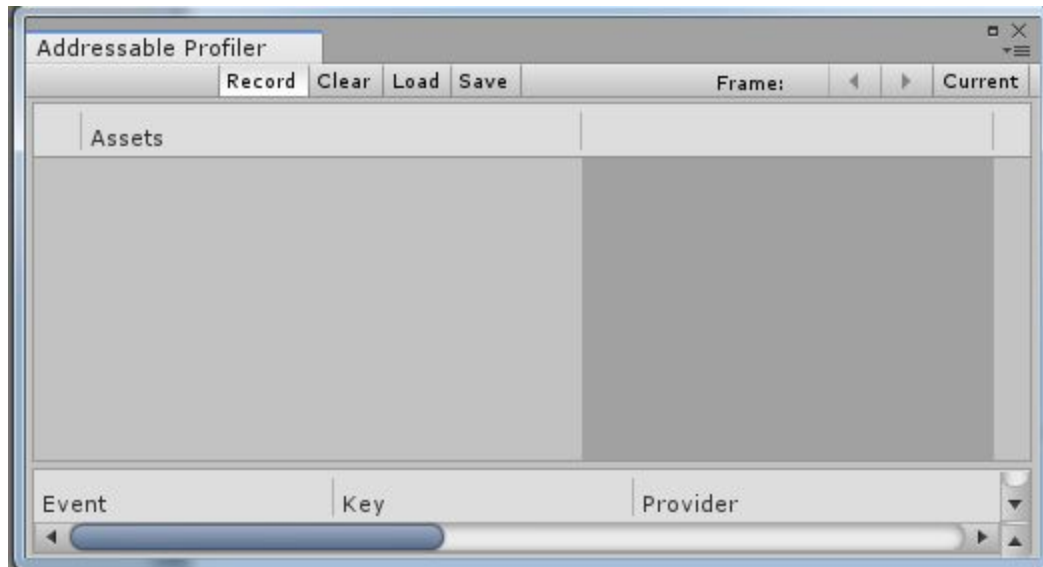
位置在: Addressables視窗上方選單, [Build / Clean](#)
清除各種暫存資料, 官方表示: 遇到奇怪的問題, 可以先試試看這個功能

@11. Built In Data

這是一個預設存在的AssetGroup, 他被設計為唯讀
會顯示專案內所有Resources資料夾的東西
可以透過這裡看到, 還有那些潛在的Resources資料夾使用中
並透過他的右鍵選單, 一鍵將這些資源搬出Resources資料夾

請小心, 經過測試, 這個AssetGroup, 可以被刪除, 但相當不建議

@12. Addressables Profiler視窗



目前這個視窗基本還是半成品, 沒什麼大用
但日後所有透過 AAS存取的資源情形, 將可以全在這邊進行檢視
使用上, 可以透過上方選單的

[Windows / Asset Management / Addressable Profiler](#)

來打開視窗本體

打開預設不會真正被啟用, 需要使用者額外去

[AddressableAssetSettings.asset](#)將 [Send Profiler Events](#)打勾

#5. 延伸閱讀

@如何寫一個可以動態刷新資源的Profile(簡單範例, 不說明)



Profile Entries	
BuildTarget	[UnityEditor.EditorUserBuildSettings.activeBuildTarget]
LocalBuildPath	Bundle/[BuildTarget]
LocalLoadPath	http://[PrivateIpAddress]:[HostingServicePort]

這是最精簡的寫法, 照這樣寫之後, Bundle包的檔案
會被放在Assets資料夾隔壁, 名為Bundle的資料夾內
並依照當前的平台名稱, 進行分類

然後預設使用, Hosting Service來提供檔案

@如何讓Hosting Service能被其他機器讀取?

簡單說, 就是要打開Windows的防火牆

以Windows 7為例:

控制台 > Windows 防火牆 > 進階設定 > 輸入規則 > 新增規則 > 連接埠 >

其他操作可能要自己google或嘗試一下, 總之就是要把 port打開

允許外部連結

然後, 請確保要連入的機器, 跟你的Hosting Service機器在**同個網域**內

這樣就可以了

@其他問題

1. IGG就找Kevin問問看
QQ號(3165461547)
2. 翻看看這份[筆記](#)有沒有寫到
3. 翻看看這幾份PPT有沒有提到
[Addressable Assets System](#) 中文
[AAS 048 Get Start](#) 中文
[AAS 048 Advance](#) 中文
4. 翻看看[官方手冊](#)(英文)
5. 爬看看[官方論壇](#)(英文)
6. 目前有可能將專案資源, 拆到別的Unity專案存放後, 進行加載使用嗎?
可以, 但步驟很繁雜, 而且使用上相當不方便, 個人不推薦使用
7. 目前有可能將多個AAS的設置, 疊加在一起嗎?
理論上可以, 035有做過這樣的實驗, 可行, 但048這個功能的關鍵API目前測試, 故障中
053本身不堪用, 沒測試過